



EMBARCADERO CONFERENCE 2025

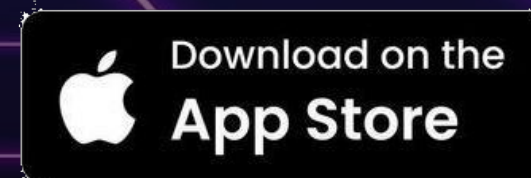
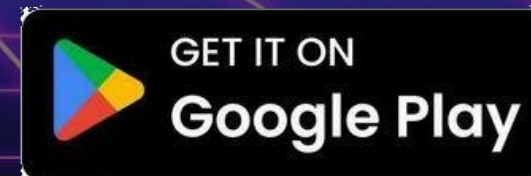
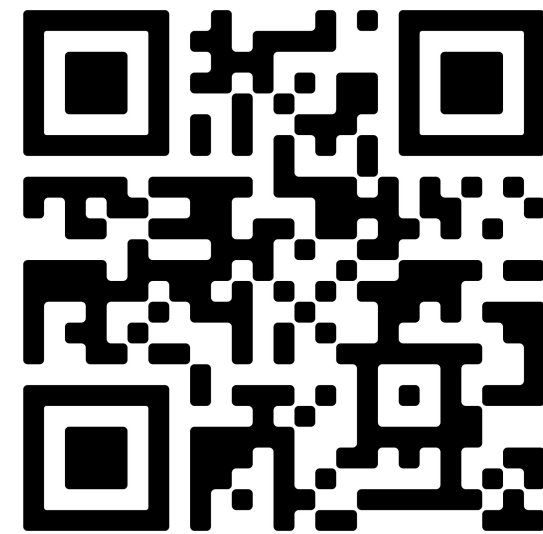
A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



Baixem o app!!!



Embarcadero Conference



Samuel Soares

- **Formação**

Técnico em Informática no Instituto Federal de Ciências e Tecnologia - IFPE
Análise e Desenvolvimento de Sistemas - Estácio.

- Desenvolvedor Delphi a **8 anos**;
- Atualmente atuando como **Analista e Desenvolvedor Sênior na TOTVS**, focando em aplicações Mobile, e APIs.



A Arte de Caçar Bugs: Debug Inteligente no Delphi

Samuel Soares

Como usar ferramentas?

O debugger não serve apenas para encontrar erros, mas para compreender o comportamento da aplicação.

Permite analisar variáveis, fluxo de execução e memória em tempo real.

Evita retrabalho e acelera a identificação de problemas.

Toda profissão tem suas ferramentas, use a sua



Debug não é sorte, é técnica

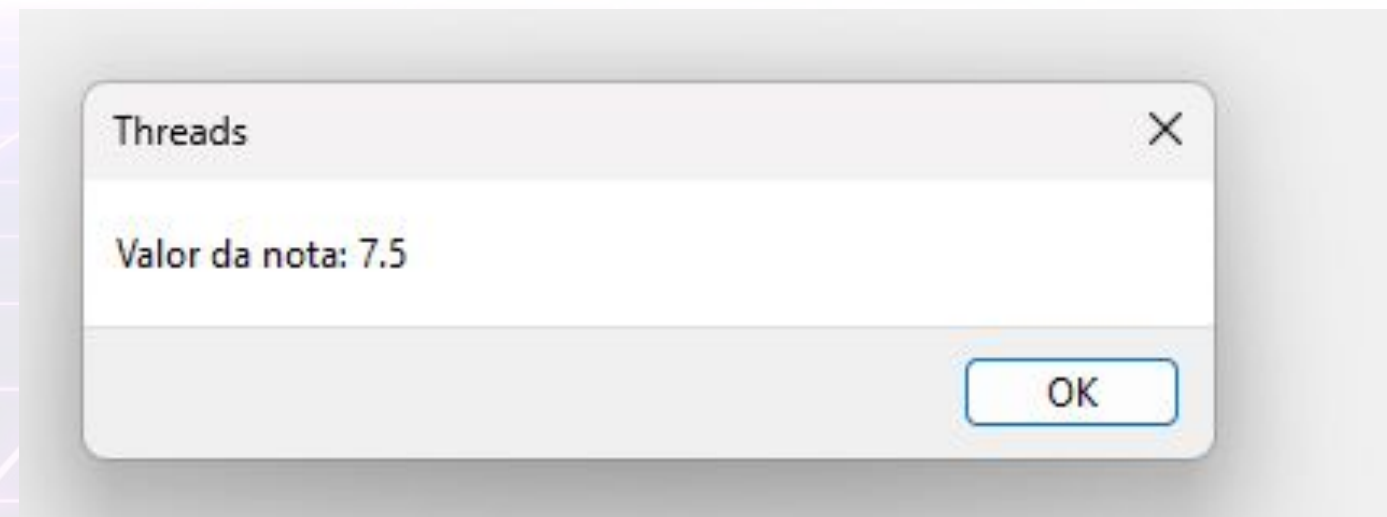


Hora de caçar os bugs.



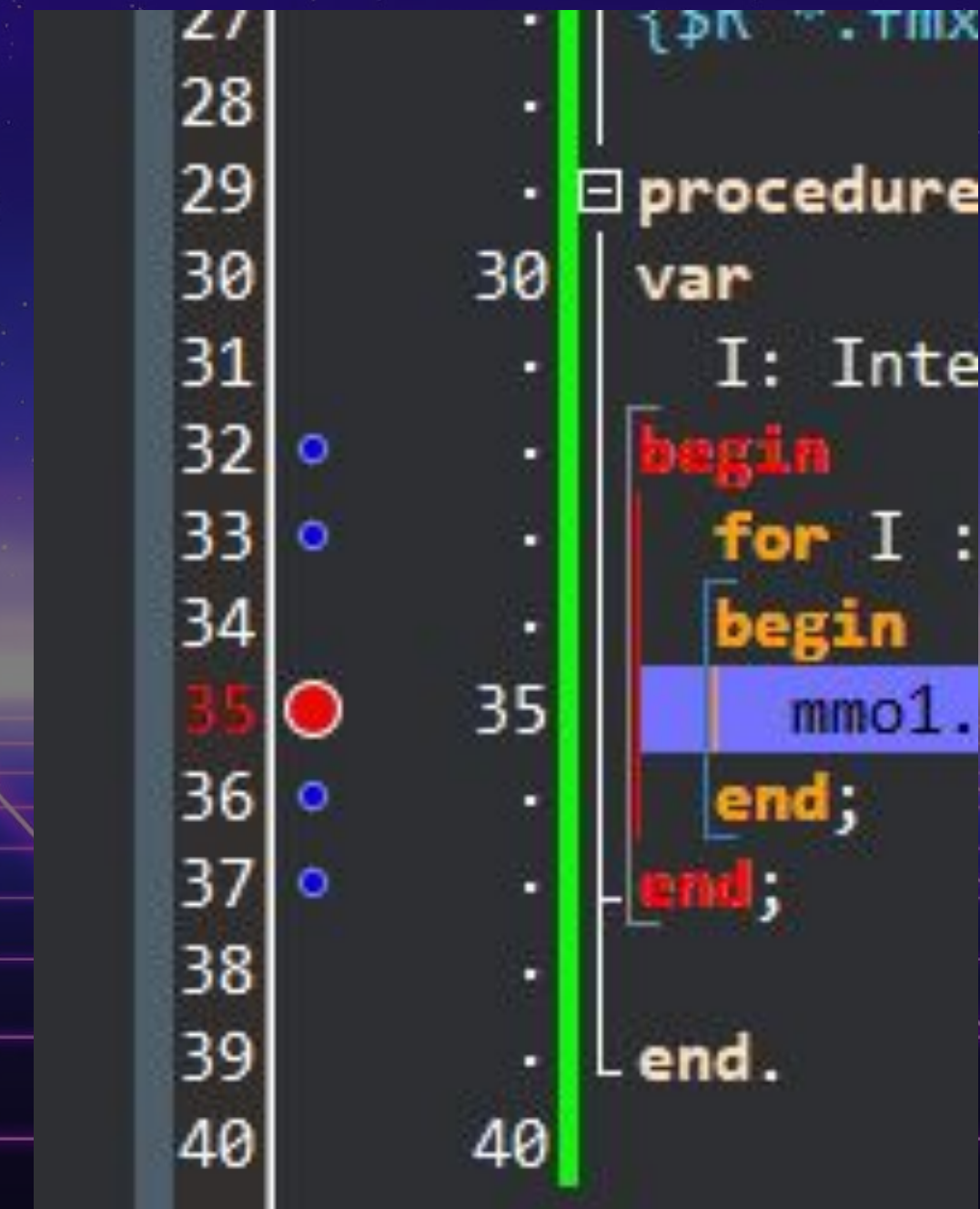
Qual o objetivo dessa palestra?

- Parar de usar o ShowMessage() como debug;
- Dominar breakpoints e watches;
- Ler o call stack como um detetive; 🕵️
- Usar a IDE como uma aliada para encontrar bugs;



Breakpoints e suas variações

- Um breakpoint é um ponto de parada que você coloca no código para pausar a execução do programa durante o debug.

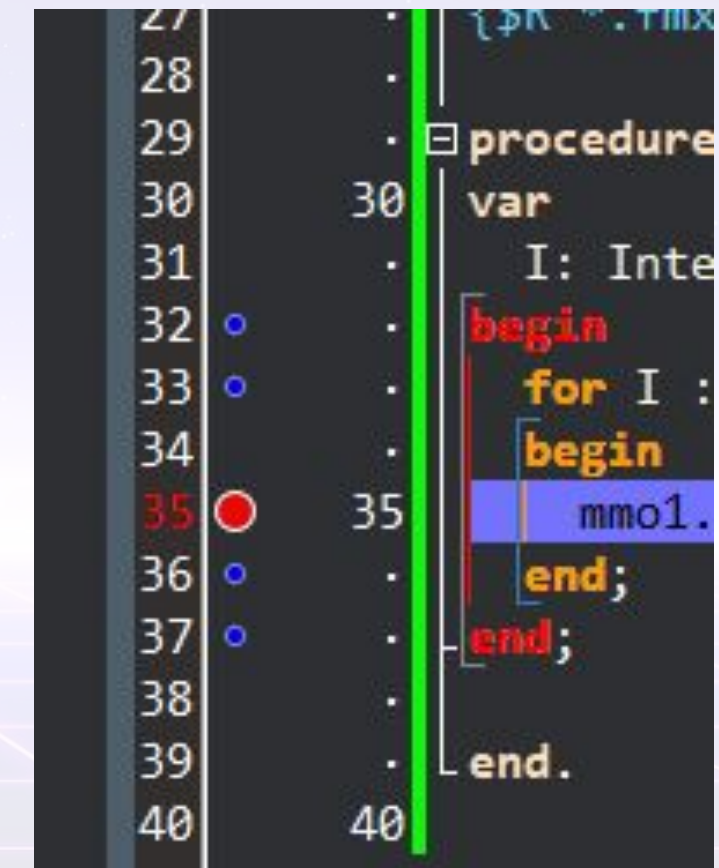
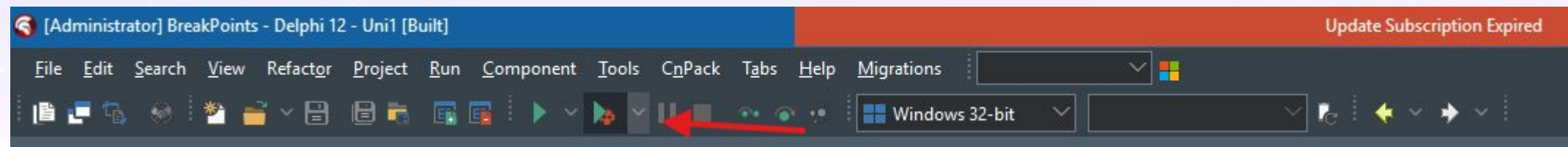


Por que o usar o Breakpoint?

- Com o breakpoint, conseguimos parar em uma linha específica;
- Conseguimos analisar os valores das variáveis em tempo real; 🖐
- Entendemos bem o fluxo do código;
- Ajuda a encontrar erros mais rápido; 🐛

Como utilizamos o breakpoint?

- Clique na margem esquerda da linha do código (aparece o ponto vermelho).
- Rode o programa em *Debug*
- O programa vai parar nessa linha selecionada



Tipos de breakpoint

- **Simples:** para a execução sempre que a linha for alcançada
- **Condicional:** só para se uma condição for verdadeira

Ex: $(i > 10)$

Ou, podemos fazer com que o breakpoint seja acionado depois de passar por ele uma determinada quantidade de vezes.

Vamos para prática?



Atalhos no breakpoint

- **F5 - Habilitar/desabilitar breakpoint**

Ativa ou desativa um breakpoint já marcado;

- **F7 - Step Into (Entrar no método)**

Avança para dentro do método ou função que está sendo chamado, permitindo analisar linha a linha;

- **F8 - Step Over (Próximo passo)**

Executa a linha atual e segue para a próxima, sem entrar em métodos/funções;

- **F9 - Run / Continue (Executar até o fim)**

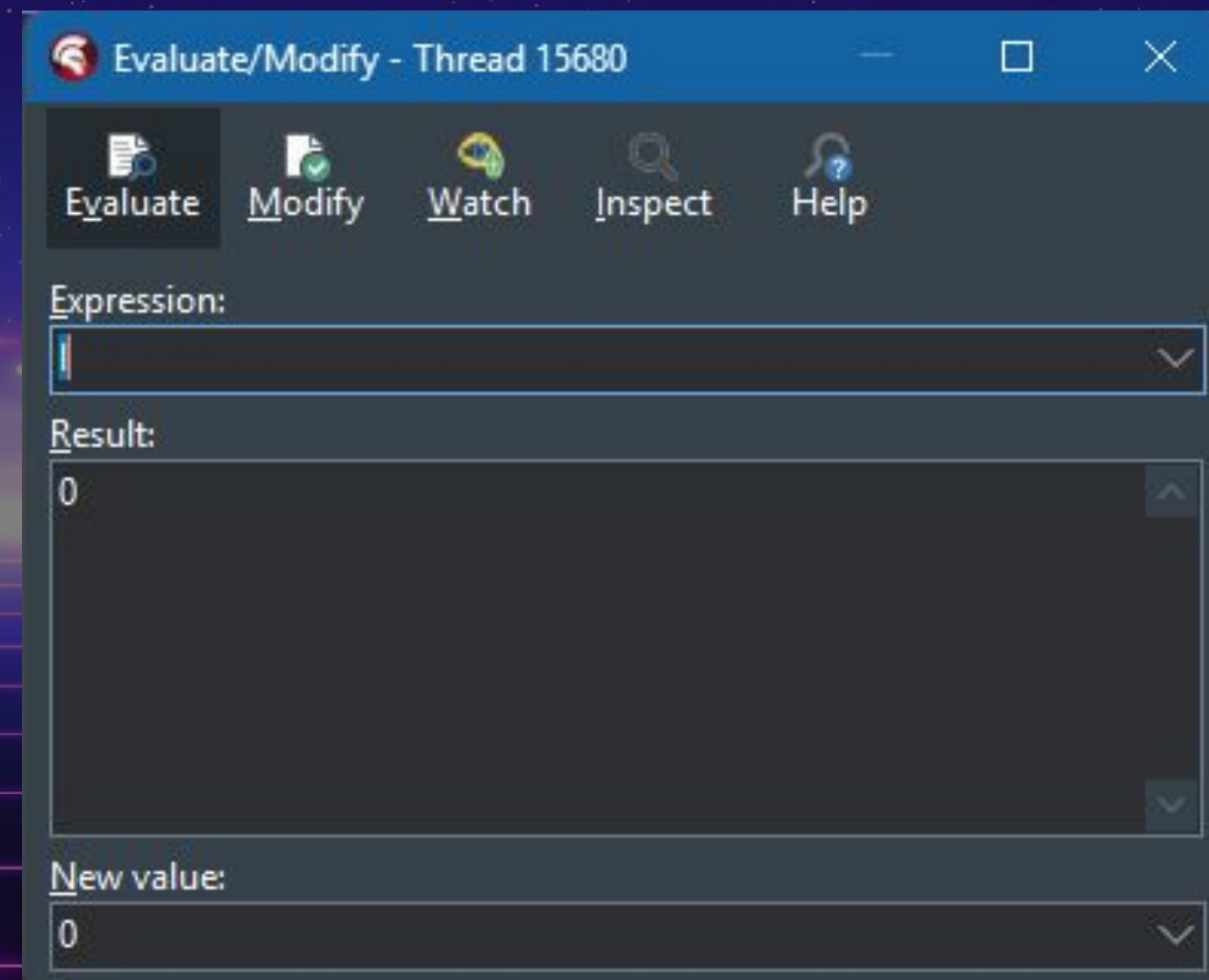
Continua a execução normal do programa até o próximo breakpoint ou até encerrar.

Programador vs Bug



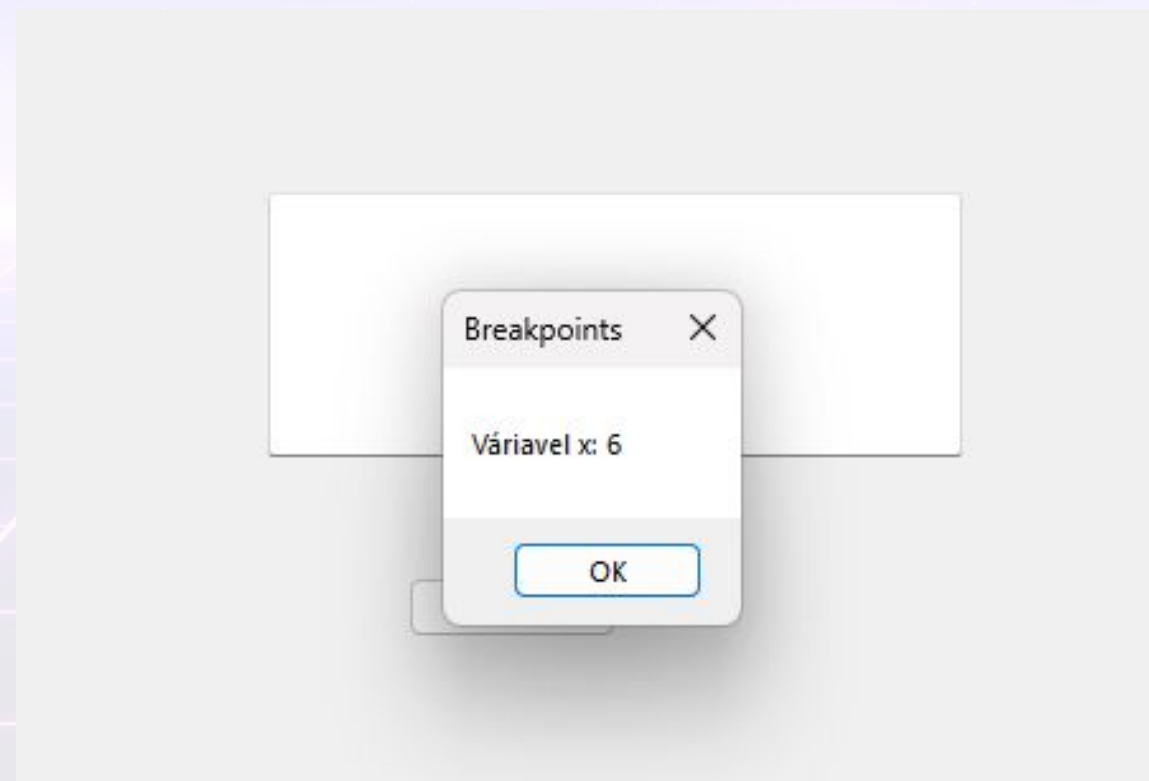
Waches e Local Variables

Permitem inspecionar o valor de variáveis locais em tempo real. Também podemos monitorar variáveis e expressões específicas escolhidas.



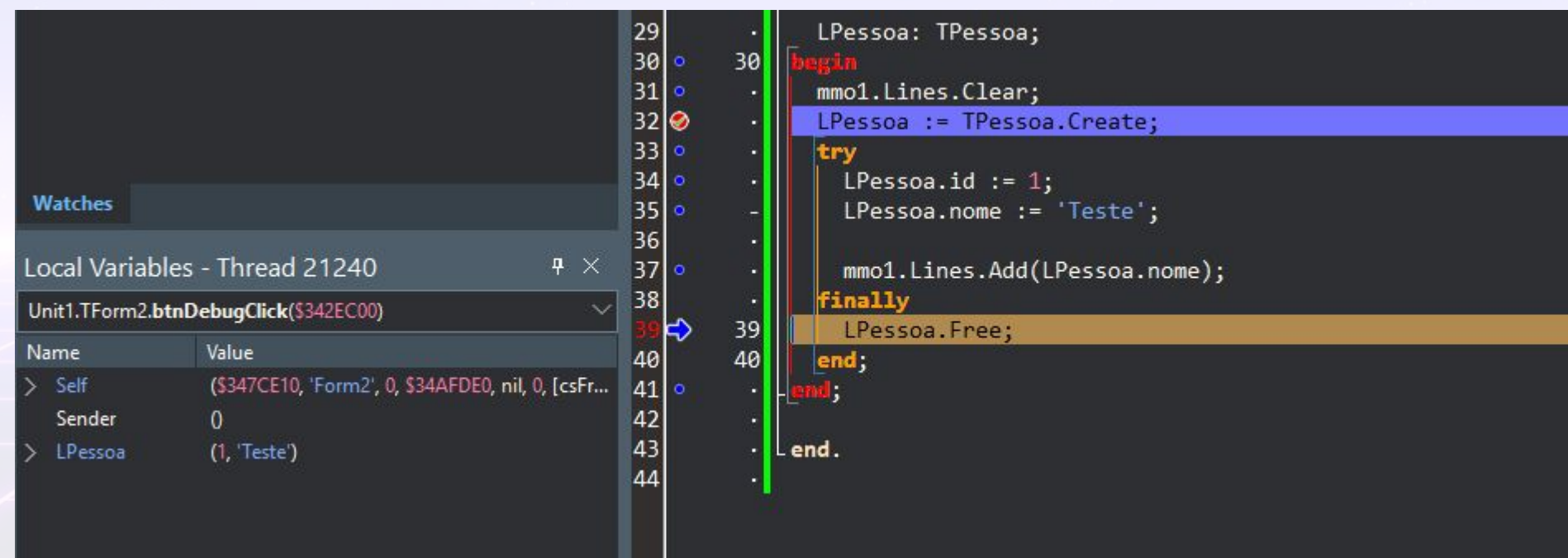
Por que utilizar o painel das Local Variable e Watches?

- Conseguimos analisar os valores das variáveis locais em tempo real;
- Monitorar expressões específicas escolhidas pelo dev;
- Ajuda a encontrar erros mais rápido; 🐛
- Alterar valores em tempo de execução;
- Evitamos isso em produção:



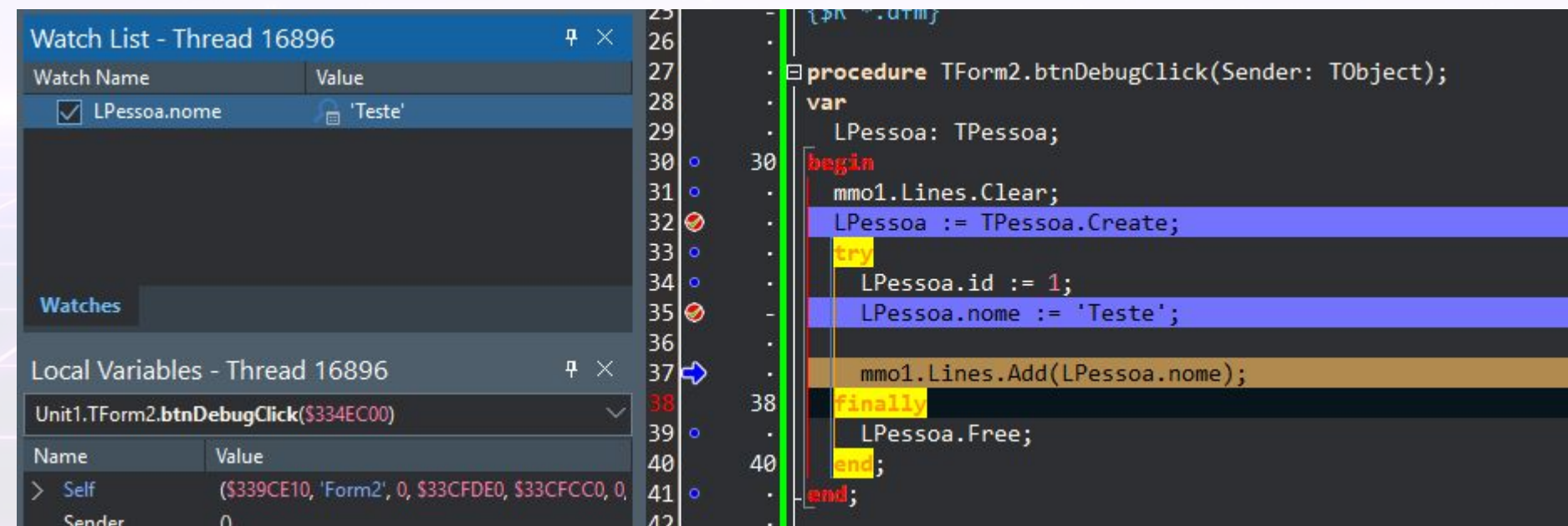
Como utilizamos a janela do LocalVariables?

- Rode o programa em *Debug*
- Quando entramos em modo de debug, a própria ide já nos exibe essa janela;



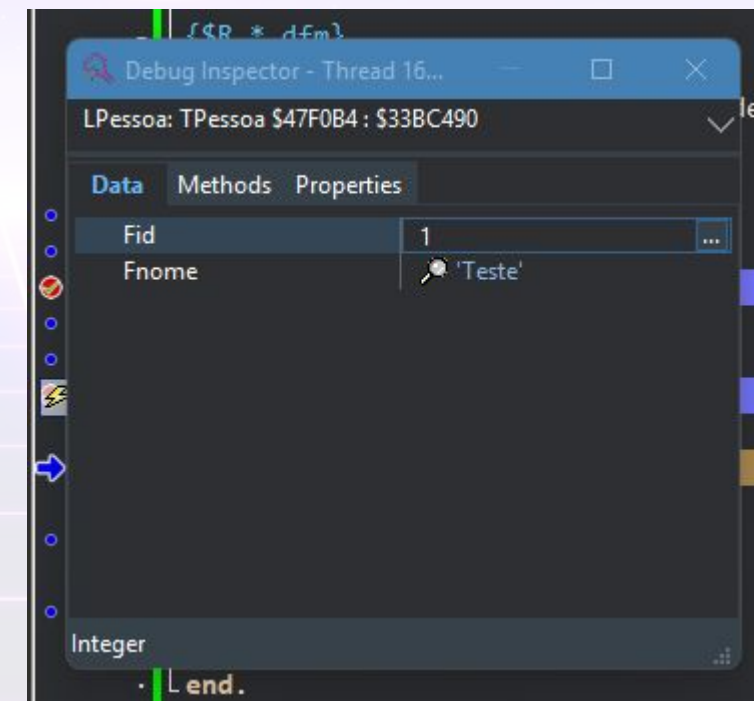
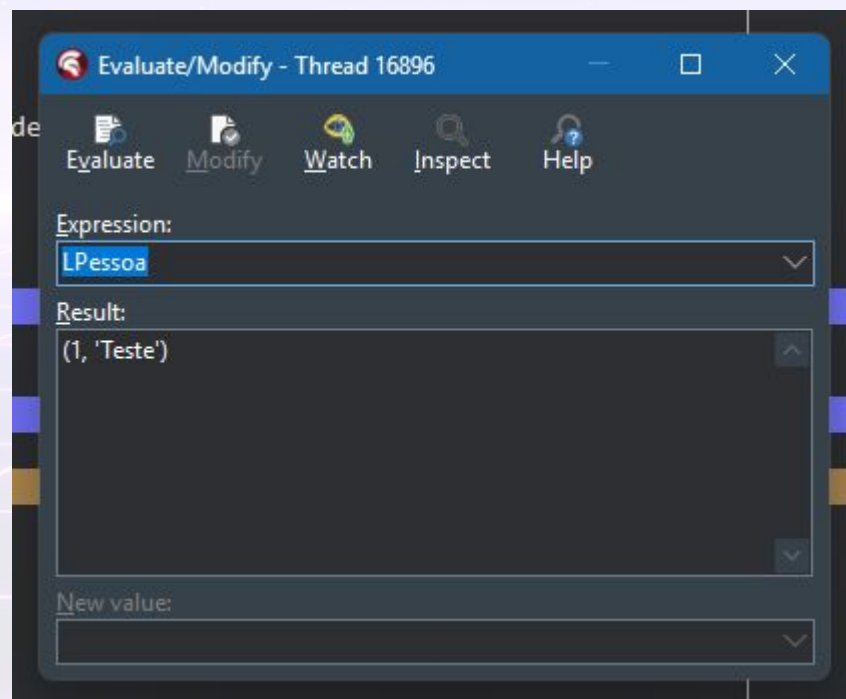
Como utilizamos a janela do Watches?

- Rode o programa em *Debug*;
- Podemos adicionar um watch em uma variável específica, e olhar o comportamento por todo o sistema;



Como utilizamos a janela do Evaluate/Modify?

- Rode o programa em *Debug*;
- Podemos alterar o valor de uma variável, para um teste específico;
- Também conseguimos olhar todos métodos que aquela variável tem acesso.



Vamos para prática?



Atalhos da IDE para investigar variáveis

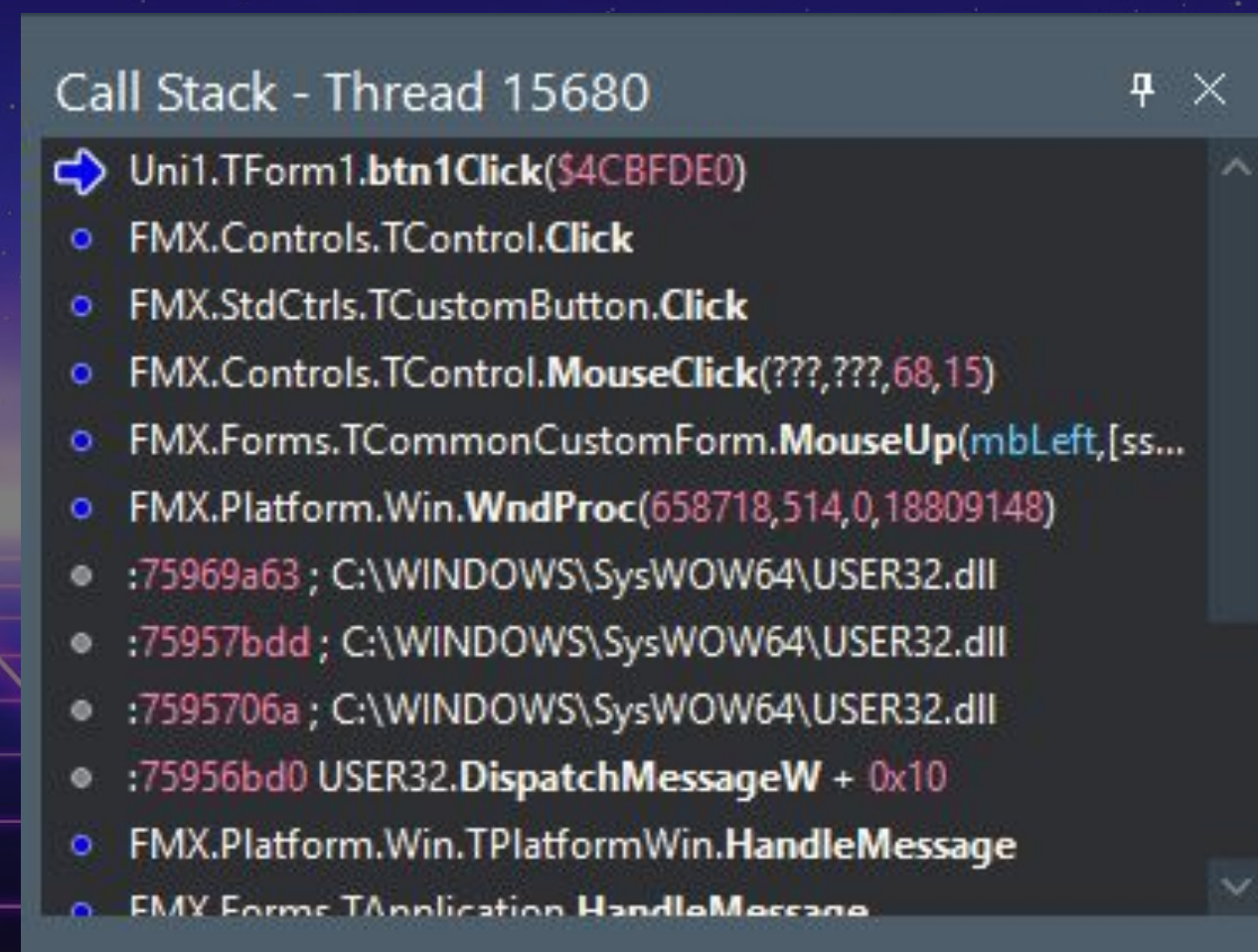
-  **Ctrl + F5 - Adicionar Watch**
Cria um Watch diretamente a partir da seleção ou expressão digitada.
-  **Ctrl + F7 - Avaliar/Modificar Expressão (Evaluate/Modify)**
Abre uma janela onde você pode digitar o nome de uma variável e pode alterar;
-  **Alt + F5 - Debug Inspector**
Abre a tela de debug inspector e consegue visualizar os detalhes da variável selecionada;

Programador vs bug



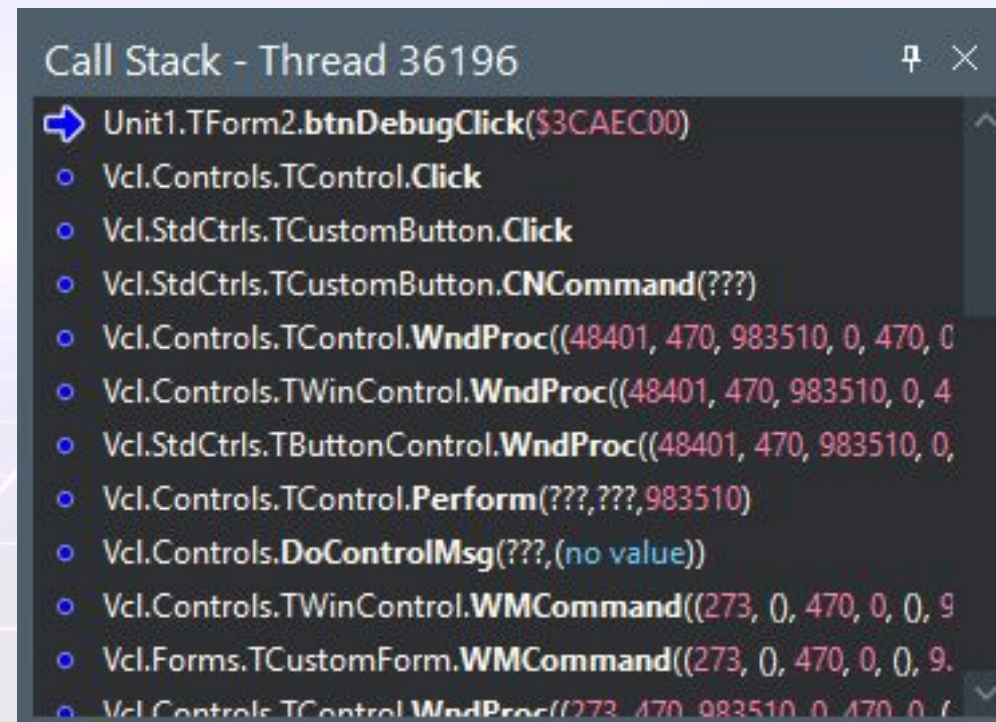
Call Stack

Lista que mostra todas as chamadas de métodos/funções que levaram até o ponto atual da execução. Funciona como uma “trilha” do programa até o breakpoint.



Por que utilizar o painel da Call Stack?

- Entender como o código chegou até ali.
- Descobrir a ordem de chamadas entre métodos.
- Ajudar a identificar loops ou recursões inesperadas.
- Localizar o ponto de origem de erros em pilhas complexas.



Como funciona na prática?

Quando o programa para em um breakpoint:

- O Delphi mostra a pilha de chamadas em ordem.
- O item no topo é onde o programa está parado agora.
- Os itens abaixo são os métodos que chamaram essa linha.

Exemplo



Vamos para prática?



Atalhos da IDE para abrir Call Stack

- 📌 **Ctrl + Alt + C → Abre a janela de Call Stack.**

Programador vs bug



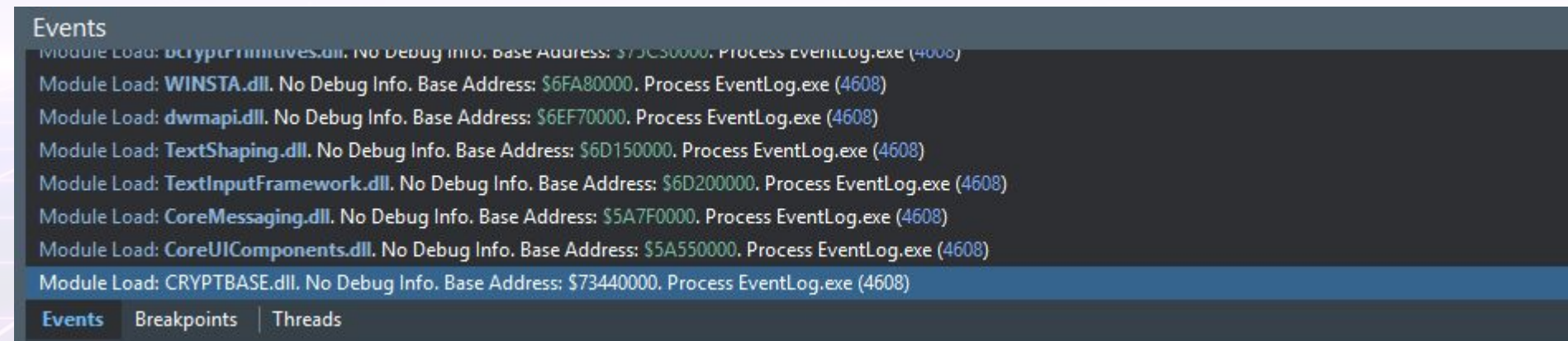
Event Log

Registra eventos e exceções em tempo real. Pode substituir o uso de ShowMessage() para acompanhar fluxos.

```
Events
Module Unload: igus9dxvab2.dll. Process BreakPoints.exe (27520)
Module Unload: igdumdim32.dll. Process BreakPoints.exe (27520)
Thread Start: Thread ID: 23268. Process BreakPoints.exe (27520)
Thread Start: Thread ID: 12256. Process BreakPoints.exe (27520)
Thread Exit: Thread ID: 12256. Process BreakPoints.exe (27520)
Thread Exit: Thread ID: 23268. Process BreakPoints.exe (27520)
Module Load: WindowsCodecs.dll. No Debug Info. Base Address: $6CA40000. Process BreakPoints.exe (27520)
Source Breakpoint at $77A0905C: C:\Users\samue\Desktop\EmbarcaderoConferenc\BreakPoints\Uni1.pas line 35. Process BreakPoints.exe (27520)
Events | Breakpoints | Threads
```


Por que utilizar o painel do Event Log?

- Acompanhar o que a aplicação e a IDE estão processando.
- Identificar exceções levantadas (mesmo quando não tratadas).
- Ver mensagens de debug personalizadas
- Auxiliar no rastreamento de problemas sem precisar alterar o código.



ATENÇÃO!!!



Vamos para prática?



Atalhos da IDE para abrir os Event Logs

- 📌 **Ctrl + Alt + V → Abre a janela de Event Logs.**

O que aprendemos?

O que aprendemos nessa palestra?

Temos muitas ferramentas para nos ajudar

- **Breakpoints** - Colocamos um ponto de parada para investigarmos;
- **Local Variables, Watches e Evaluate/Modify** - Colocamos uma lupa para investigar como cada ponto do nosso código se comporta;
- **Call Stack** - Olhamos com calma onde todo o nosso código passou e seguimos as pegadas do bug;
- **Event Logs** - Analisamos a fundo como foi cada passo do nosso programa, e lemos as mensagens completas de erros;

O que aprendemos nessa palestra?

Mas será que isso é suficiente?

- Não, **precisamos usar** as ferramentas que temos a disposição.



Dúvidas e sugestões



embarcadero®

EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



Quer me ver na
#ECON26?
Acesse o QRCode e
avale minha palestra!



Samuel Soares



@samuelsoarees



<https://www.linkedin.com/in/samuel-soares-403193123/>



samuelsoares179@gmail.com



(87) 9 8111-2372